



TextilBot

Un Chatbot RAG pour la Conformité Réglementaire Textile

Réalisé par :

Fatima-Ezzahra Joulal

Mariam El Kanfoudi

Oussama Taibouche

Table des matières

Contents

Table des matières	2
1. Introduction and context	4
1.1 General context.....	4
1.2 Problematic	4
1.3 Proposed solution	4
2. Objectives and specifications.....	5
2.1 Functional objectives	5
2.2 Non-functional objectives.....	5
2.3 Constraints	5
3. Overall system architecture	5
3.1 Overview	5
3.2 Description of the flows.....	6
4. Technological choices	6
4.1 Choice of Embedding.....	6
4.2 Choice of LLM	7
5. Modélisation UML.....	8
5.1 Diagramme de cas d'utilisation.....	8
5.2 Diagramme de séquence	8
5.3 Diagramme de classes (vue logique des modules)	9
6. Detailed description of the modules.....	9
6.1 module ingest.py by Fatima-Ezzahra Joulal.....	9
6.2 Module rag.py by Mariam El Kanfoudi	10
6.3 Module app.py by Oussama Taibouche.....	12
7. Difficultés rencontrées et solutions	13
7.1 Connexion à Qdrant Cloud.....	13
7.2 Cohérence de la langue de réponse	13
7.3 Greeting Management.....	14
8. Tests & validation.....	14
9. Task allocation and project management.....	14
10. Conclusion.....	15
TextilBot:	16
A Retrieval-Augmented Generation Chatbot for Textile Regulatory Compliance	16
Abstract.....	16
1. Introduction	17
2. State of the Art.....	17

3. Contribution	18
3.1 System Architecture.....	18
3.2 Cross-Lingual Consistency Problem and Solution	18
3.3 LLM Selection and Migration	19
4. Results and Discussion	19
5. Conclusion.....	20
6. References	20

PART 1: DETAILED DESCRIPTION OF THE SOLUTION

1. Introduction and context

1.1 General context

The European textile industry has to follow a lot of rules for labeling products. These rules are laid out in a document called Regulation (EU) No 1007/2011 from the European Parliament and the Council. It is really long with 64 pages including all the technical parts. It tells manufacturers, importers and distributors what they have to do: what names to use for different fibers, how much of each fiber can be in a product, which products do not have to follow these rules and how to label products in multiple languages.

People who work in the textile industry, like quality managers, designers, lawyers and people who check if products meet the rules have to read this document a lot to make sure products are okay to sell. Reading this document takes time and it is easy to misunderstand what it says. You have to understand the legal language that the European Union uses. The European textile. The Regulation (EU) No 1007/2011 are very important for these people to know about.

1.2 Problematic

How can someone without a background quickly get a reliable and easy-to-understand answer to a question about textile compliance?

Our solution comes to replace traditional methods, like searching for keywords in PDFs without a clear source of the information they just lose a great time in reading whole documents without getting clear answers, so our application TextilBot comes to make a difference, saving time and reduce stress, getting easy answers with their sources and make the fastest decisions

1.3 Proposed solution

TextilBot addresses this issue with a Retrieval-Augmented Generation (RAG) architecture: the regulatory text is cut, vectorized and indexed in a vector database; each user question is then semantically matched with the most relevant passages, which are injected as context into a large language model (LLM) responsible for formulating a precise, sourced answer in the language of the question.

2. Objectives and specifications

2.1 Functional objectives

- Users can ask questions in French or English about the EU Regulation 1007/2011.
- Gives a clear answer, with the exact source (Article, Annex).
- Can understand what language the user is using and responds in the same one.
- Offer options of saving, deleting the conversation and let the user give feedbacks about the answers.

2.2 Non-functional objectives

- Acceptable response time for interactive use (a few seconds).
- Modular architecture (separate ingestion, retrieval, generation, and interface).
- Web-based deployment (Streamlit Cloud).

2.3 Constraints

The choice of tools was dictated by cost constraints (free or limited for students). The chosen solution combines:

- Vector database: Qdrant Cloud (free option);
- LLM: API access (initially Groq/Llama, eventually migrated to Gemini);
- Hosting : Streamlit Community Cloud;

3. Overall system architecture

3.1 Overview

The system architecture is based on two pipelines: a preparation pipeline and an inference pipeline.

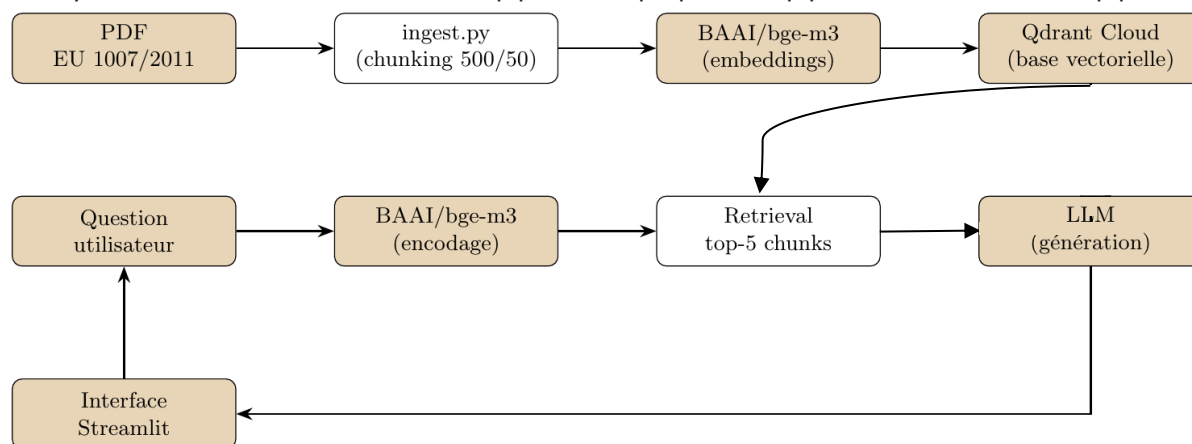


Figure 1 — Architecture générale du pipeline RAG de TextilBot

3.2 Description of the flows

1. Ingestion: The official PDF of the regulations is extracted and split into 500-word chunks with a 50-word overlap to preserve context at boundaries. Each chunk is then transformed into an embedding vector and stored in Qdrant Cloud with its metadata (source text, document reference).
2. Query: The user's question is vectorized using the same embedding model and then compared using cosine similarity to the stored vectors to extract the five most relevant chunks. These chunks constitute the context provided to the LLM, which generates the final answer.
3. Presentation: The answer is displayed in the Streamlit interface, with management of the conversation history, export, and user feedback.

4. Technological choices

Composant	Technologie	Justification
Embeddings	BAAI/bge-m3	Modèle multilingue open-source, performant sur le français et l'anglais, exécutable localement sans coût d'API.
Base vectorielle	Qdrant Cloud	Offre gratuite suffisante pour le volume du projet (68 chunks), API simple, hébergement géré.
LLM	Gemini 2.5 Flash	Migration effectuée pour des raisons de coût, de latence et de disponibilité de quota gratuit plus généreux.
Interface	Streamlit	Framework Python permettant un développement rapide d'interface conversationnelle, déploiement gratuit via Streamlit Community Cloud.
Extraction PDF	pypdf	Bibliothèque légère, suffisante pour un PDF textuel (non scanné).

4.1 Choice of Embedding

For the vectorization of chunks and user questions, the BAAI/bge-m3 model was selected from among several alternatives. This model stands out for its native multilingual capability, supporting more than 100 languages, which is particularly well-suited to TextilBot, which processes questions in French and English.

BAAI/bge-m3 generates 1024-dimensional vectors offering a rich semantic representation, surpassing alternatives. It is better than models like all-MiniLM-L6-v2, which is only 384 dimensions and not as good or OpenAI's text-embedding-ada-002 which you have to pay for. It ranks among the top models on the Massive Text Embedding Benchmark for semantic search tasks and anyone can get it for free from HuggingFace without needing a special API key.

Modèle	Dimensions	Multilingue	Précision sémantique	Textes juridiques	Coût	API requise
all-MiniLM-L6-v2	384 <i>faible</i>	✗ EN seulement	●●○	●○○	Gratuit	Non
text-embedding-ada-002 (OpenAI)	1536	✓ 100+ langues	●●●	●●●	Payant	Oui
multilingual-e5-large	1024	✓ 100+ langues	●●○	●●○	Gratuit	Non
BAAI/bge-m3 ✓ Choix final	1024	✓ 100+ langues	●●●	●●●	Gratuit	Non

● = bon | ○ = faible | BAAI/bge-m3 : meilleur score MTEB pour la recherche sémantique multilingue

4.2 Choice of LLM

Several models were evaluated for generating responses. Llama 3.3 70B via Groq was initially used due to its free availability and high inference speed thanks to its LPU chips. However, during testing, the limit of 100,000 tokens per day was quickly reached, rendering the system unavailable for several tens of minutes.

We therefore migrated to Google's Gemini 2.5 Flash, which offers a significantly higher free limit (over one million tokens per day), excellent response quality in both French and English, and simple API integration. This choice guarantees the chatbot's continuous availability without quota-related interruptions, which is essential for a regulatory compliance assistant used in production.

Modèle	Limite tokens/jour	Vitesse	Qualité FR	Coût	Multilingue	API facile
Llama 3.3 70B (Groq)	100 000 <i>limité</i> ⚠	●●●	●●○	Gratuit	✓	✓
GPT-4o (OpenAI)	Selon crédit \$	●●●	●●●	Payant	✓	✓
Mistral Large (Mistral AI)	Selon crédit \$	●●○	●●●	Payant	✓	✓
Claude 3.5 Sonnet (Anthropic)	Selon crédit \$	●●●	●●●	Payant	✓	✓
Gemini 2.5 Flash (Google) ✓ Choix final	1 000 000+	●●●	●●●	Gratuit	✓	✓

● = bon | ○ = faible | Llama éliminé : limite 100k tokens/jour dépassée pendant les tests

5. Modélisation UML

5.1 Diagramme de cas d'utilisation

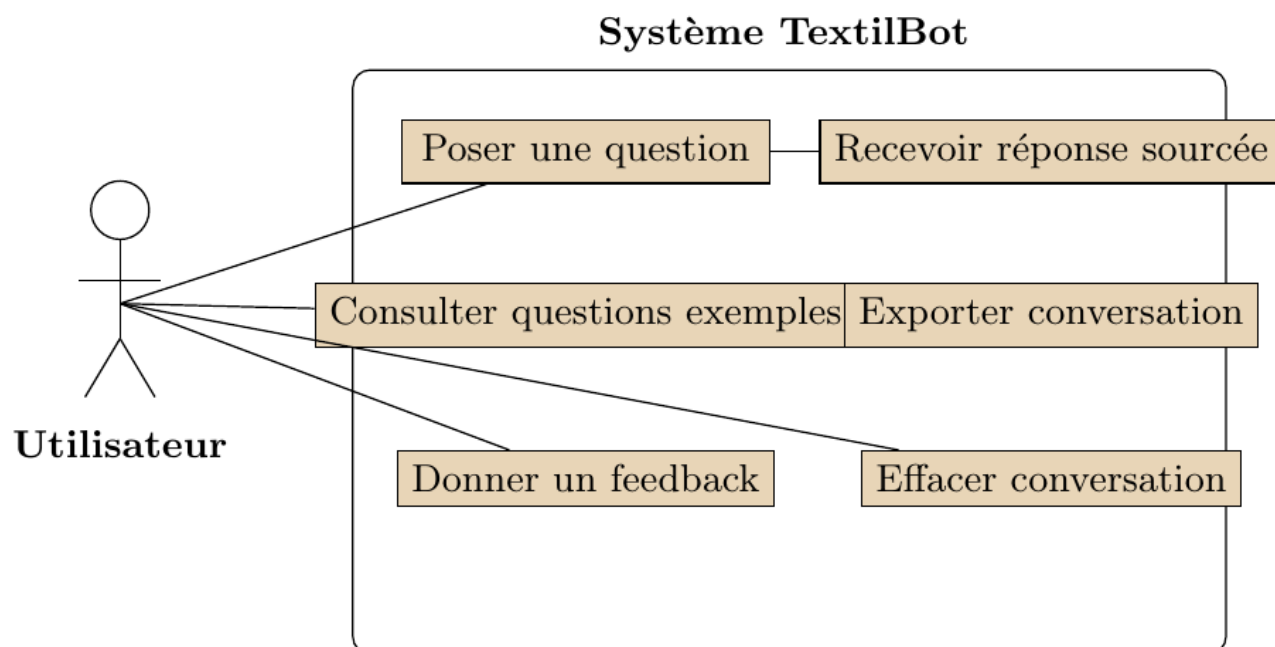


Figure 2 — Diagramme de cas d'utilisation de TextilBot

5.2 Diagramme de séquence

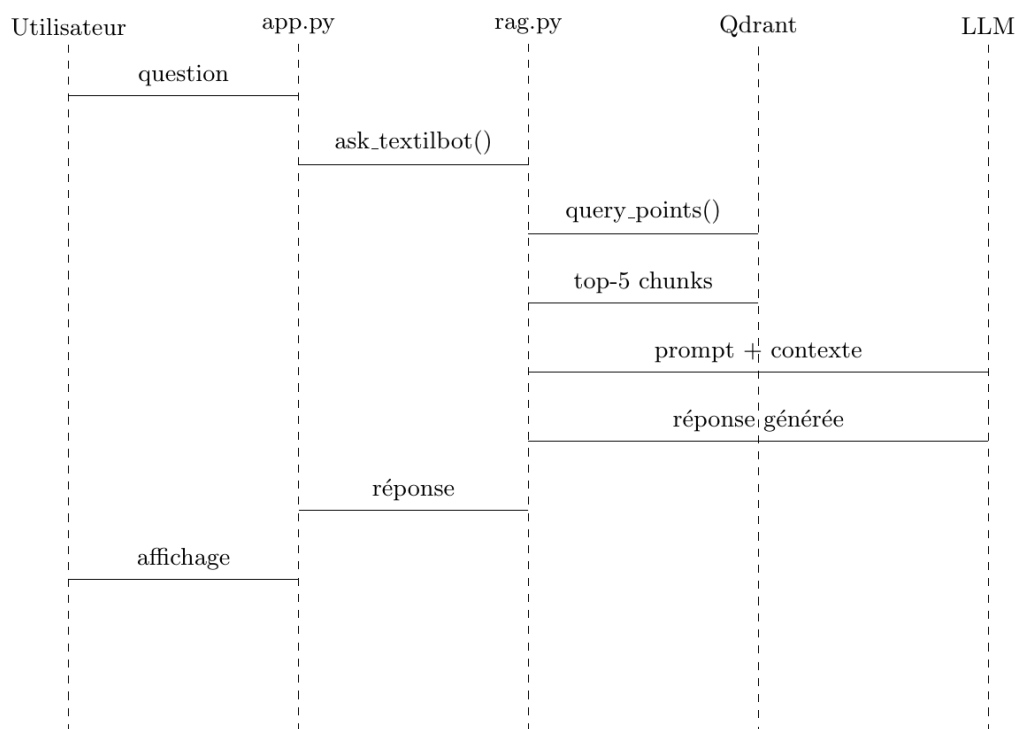


Figure 3 — Traitement d'une question utilisateur

5.3 Diagramme de classes (vue logique des modules)

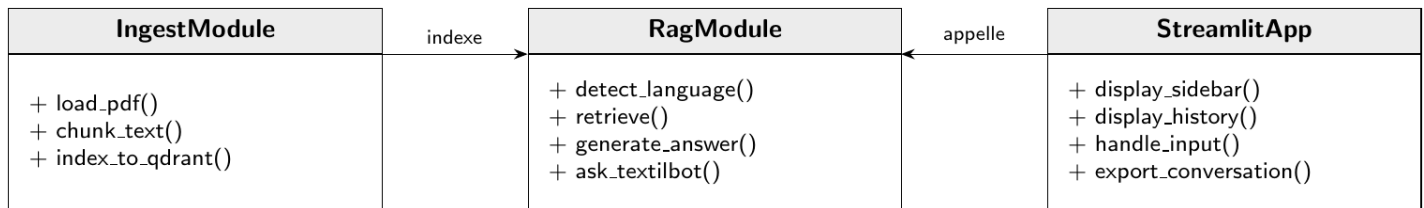


Figure 4 — Vue logique des trois modules principaux

6. 6. Detailed description of the modules

6.1 module ingest.py by Fatima-Ezzahra Joulal

This module is responsible for preparing the data:

- `load_pdf()` : extracting the raw text from the official PDF via pypdf
- `chunk_text()` : cutting the text into segments of 500 words with an overlap of 50 words, producing 68 chunks in total. This overlap limits the loss of information when a regulatory concept extends across the border between two segments.;
- Indexing of each chunk in Qdrant Cloud with its embedding and metadata (text, source).

```

1  import os
2  from dotenv import load_dotenv
3  from pypdf import PdfReader
4  from sentence_transformers import SentenceTransformer
5  from qdrant_client import QdrantClient
6  from qdrant_client.models import Distance, VectorParams, PointStruct
7  from tqdm import tqdm

```

```

9  load_dotenv()
10 QDRANT_URL = os.getenv("QDRANT_URL")
11 QDRANT_API_KEY = os.getenv("QDRANT_API_KEY")
12
13 COLLECTION_NAME = os.getenv("COLLECTION_NAME")
14
15 # 1. Charger Le PDF
16 def load_pdf(filepath):
17     reader = PdfReader(filepath)
18     text = ""
19     for page in reader.pages:
20         extracted = page.extract_text()
21         if extracted:
22             text += extracted + "\n"
23     return text

```

```

25 # 2. Chunker
26 def chunk_text(text, source, chunk_size=500, overlap=50):
27     words = text.split()
28     chunks = []
29     i = 0
30     chunk_id = 0
31     while i < len(words):
32         chunk_words = words[i:i + chunk_size]
33         chunk_str = " ".join(chunk_words)
34         if len(chunk_words) > 30:
35             chunks.append({
36                 "id": f"{source}_{chunk_id}",
37                 "text": chunk_str,
38                 "source": source,
39                 "chunk_index": chunk_id
40             })
41             chunk_id += 1
42             i += chunk_size - overlap
43     return chunks
44
75 print("⌚ Indexation dans Qdrant...")
76 for i in tqdm(range(0, len(chunks), 100)):
77     batch_chunks = chunks[i:i+100]
78     batch_emb = embeddings[i:i+100]
79     points = [
80         PointStruct(
81             id=i+j,
82             vector=batch_emb[j].tolist(),
83             payload={
84                 "text": batch_chunks[j]["text"],
85                 "source": batch_chunks[j]["source"],
86                 "chunk_index": batch_chunks[j]["chunk_index"]
87             }
88         )
89         for j in range(len(batch_chunks))
90     ]
91     client.upsert(collection_name=COLLECTION_NAME, points=points)
92
93 print("Indexation terminée !")

```

6.2 Module rag.py by Mariam El Kanfoudi

This module forms the core of the question-answering pipeline. The `retrieve()` function encodes the question, queries Qdrant, and returns the five closest matches.

The `generate\_answer()` function constructs the prompt sent to the Language Learning Module (LLM) by incorporating the retrieved document context and an explicit language instruction, calculated by the code (and not left solely to the model's discretion) based on the question's

language detection ('langdetect'). This approach was chosen after empirical observation that a simple text prompt such as "answer in the language of the question" in the system prompt was not always respected by the LLM when the retrieved document context was in a different language (see section 8).

The `ask_textilbot()` function orchestrates the entire process: it prioritizes simple greetings (direct responses without calling the LLM or retrieving the answer), then executes the complete retrieval → generation pipeline, and includes a mechanism for double-checking the language of the generated answer before returning it to the user.

```

1  import os
2  from dotenv import load_dotenv
3  from sentence_transformers import SentenceTransformer
4  from qdrant_client import QdrantClient
5  import google.generativeai as genai
6  from langdetect import detect, DetectorFactory
7
8  DetectorFactory.seed = 0
9
10 load_dotenv()
11
12 COLLECTION_NAME = os.getenv("COLLECTION_NAME")
13 GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
14 QDRANT_URL = os.getenv("QDRANT_URL")
15 QDRANT_API_KEY = os.getenv("QDRANT_API_KEY")
16 qdrant = QdrantClient(url=QDRANT_URL, api_key=QDRANT_API_KEY)
17 model = SentenceTransformer("BAAI/bge-m3")
18 genai.configure(api_key=GEMINI_API_KEY)
19 gemini = genai.GenerativeModel("gemini-2.5-flash")
20
21 GREETINGS = {
22     "hi", "hello", "hey", "yo", "good morning", "good evening",
23     "bonjour", "salut", "coucou", "bonsoir", "hello there"
24 }
25
26
27 def detect_language(text: str) -> str:
28     english_greetings = {"hi", "hello", "hey", "yo",
29                          "good morning", "good evening",
30                          "hello there"}
31     if text.strip().lower() in english_greetings:
32         return "en"
33     try:
34         lang = detect(text)
35         return "en" if lang == "en" else "fr"
36     except Exception:
37         return "fr"
38

```

```

40 def retrieve(question, top_k=5):
41     embedding = model.encode(question, normalize_embeddings=True)
42     results = qdrant.query_points(
43         collection_name=COLLECTION_NAME,
44         query=embedding.tolist(),
45         limit=top_k
46     ).points
47     return [
48         {"text": r.payload["text"], "source": r.payload["source"]},
49         {"score": r.score}
50     for r in results
51 ]

```

And that's the prompt we used

```

59 prompt = f"""Tu es TextilBot, un assistant expert en conformité réglementaire textile.
60 Tu réponds aux questions sur les normes textiles en te basant uniquement sur les documents fournis.
61
62 RÈGLE ABSOLUE : Tu dois répondre entièrement en {lang_tag}. Ne mélange pas les langues.
63
64 Règles :
65 - Cite toujours la source (article ou annexe)
66 - Si la question n'est pas liée aux textiles, réponds poliment en une phrase courte
67 - Sois précis et professionnel
68
69 Contexte documentaire :
70 {context}
71
72 Question : {question}
73
74 Réponds en {lang_tag} en citant les sources."""
75

```

6.3 Module app.py by Oussama Taibouche

This module provides the user interface via Streamlit:

- A graphic design reminiscent of textile weaving (brown/beige gradients, crosshatch patterns)
- A sidebar with clickable example questions, in French and English
- A time-stamped .txt export of the conversation
- A user feedback system (thumbs up/thumbs down) under each response, stored in the session;
- A button to reset the conversation.

7. Difficultés rencontrées et solutions

7.1 Connexion à Qdrant Cloud

A network connectivity issue (likely related to a firewall or restrictive proxy on the network being used) temporarily prevented access to Qdrant Cloud from the development workstation. The workaround involved switching to a 4G connection, confirming that the problem originated from a network issue rather than an application configuration problem.

We temporarily migrated to local Qdrant, which stores vectors directly on disk in a `qdrant_storage/` folder without requiring an internet connection.

```
client = QdrantClient(path="./qdrant_storage")
```

This solution allowed us to continue development and validate the complete RAG pipeline. However, local Qdrant has a major limitation: since the vectors are stored on the local PC, the application cannot be deployed online or accessed from another device.

After several attempts, we successfully established a connection with Qdrant Cloud by switching networks (to 4G), which confirmed that the problem was related to restrictions on the Wi-Fi network being used. The cloud connection was successfully restored.

```
client = QdrantClient(url=QDRANT_URL, api_key=QDRANT_API_KEY)
```

7.2 Cohérence de la langue de réponse

The most significant problem encountered was the occasional failure of the Language Management System (LMS) to adhere to the language requirement: while the system prompt requested a response in the language of the question, the model sometimes responded in French even to a question posed in English, particularly when the retrieved document context (in French) dominated the prompt.

The chosen solution involved three levels:

1. Explicit detection of the question's language by the code (`langdetect`), independent of the LMS.
2. Injection of an explicit tag [REQUIRED RESPONSE LANGUAGE: ...] into the user prompt, repeated and reinforced rather than simply left as a general rule among others.
3. Post-generation verification: the language of the generated response is re-analyzed, and in case of discrepancy, a new attempt (followed by a fallback translation) is automatically triggered.

This "detection-injection-verification" approach has proven to be more robust than a simple instruction in natural language, because it does not rely solely on the model's ability to respect one rule among others in a long prompt.

7.3 Greeting Management

Similarly, a single text instruction ("for greetings, simply reply without mentioning documents") was not always reliably followed. The solution was to handle this case upstream of the LLM, using a simple string comparison, ensuring deterministic behavior and avoiding an unnecessary call to the retrieval and generation pipeline.

8. Tests & validation

System validation relied on a set of test questions covering:

- direct factual questions (e.g., list of products exempt from labeling, Annex V)
- regulatory calculation questions (e.g., foreign fiber tolerances, Article 20)
- off-topic questions (validation of polite refusal behavior)
- simple greetings
- identical questions asked in French and then in English, to validate the consistency of the response language.

The responses were manually compared to the source text of the regulation to verify the absence of hallucinations and the accuracy of the cited references.

9. Task allocation and project management

Membre	Contribution
Fatima-Ezzahra Joulal	Ingestion et indexation du document (ingest.py), préparation des chunks.
Mariam El Kanfoudi	Conception et implémentation du pipeline RAG (rag.py), gestion multilingue, intégration LLM.
Oussama Taibouche	Conception et développement de l'interface utilisateur (app.py), design, déploiement.

The source code is publicly versioned on GitHub (mariamkanfoudi/Textilbot), with a commit history structured by functionality.

10. Conclusion

TextilBot démontre la faisabilité d'un assistant de conformité réglementaire textile basé sur une architecture RAG, capable de répondre de façon précise, sourcée et multilingue à des questions portant sur le règlement EU 1007/2011. Le projet a permis de mettre en pratique l'ensemble de la chaîne d'un système RAG : extraction et découpage de documents, embeddings, base vectorielle, génération augmentée, et restitution via une interface conviviale.

PART 2 — SCIENTIFIC REPORT

TextilBot:

A Retrieval-Augmented Generation Chatbot for Textile Regulatory Compliance

Abstract

The textile industry has to follow a lot of rules. This means people have to read documents like Regulation (EU) No 1007/2011 on textile fibre names and labelling. It is a time-consuming and costing.

This paper is about TextilBot, a system that helps answer questions about EU 1007/2011 in French and English. TextilBot gives answers that are based on sources. It tells you where it got the information from. The TextilBot system is really helpful because it provides information about the textile industry. The TextilBot system uses a tools like special sentence embeddings from BAAI/bge-m3, a database from Qdrant Cloud and a big language model called Gemini 2.5 Flash. All of these tools are connected through a user interface called Streamlit. The Streamlit interface makes it easy to use the TextilBot system. When we were building the TextilBot system we found a problem. The problem happened when the language of the answer was different from the language of the question. We fixed this problem by using a strategy that detects the language. Then puts the right language into the answer. This strategy helps the TextilBot system give answers. We tested the TextilBot system with some questions that are commonly asked about following the rules in the textile industry. The answers we got from the TextilBot system were accurate. They matched the language of the question. This is really good because it means the TextilBot system is working well.

We know that the TextilBot system is not perfect yet. For example the TextilBot system can only answer questions based on one document, which's Regulation (EU) No 1007/2011. We want to make the TextilBot system better, in the future by adding documents and coming up with new ways to measure how well the TextilBot system is working. This will help the textile industry a lot.

1. Introduction

For companies in the textile sector, regulatory compliance is so important that it can become a tedious operational task. To alleviate this recurring burden, our project, Textilbot, aims to design an intelligent conversational assistant capable of providing real information on regulatory compliance for textile professionals, delivering instant and accurate answers. Textilbot leverages the advanced capabilities of Large Language Models (LLMs). Unlike a keyword search engine, LLMs allow Textilbot to understand users' natural language and generate contextualized responses by drawing on relevant legislation. Thanks to its RAG architecture, the LLM doesn't simply guess answers; it retrieves information from a regulatory database, thus guaranteeing sourced responses while minimizing the risk of inaccuracies and hallucinations.

2. State of the Art

Retrieval-Augmented Generation was introduced as a method to ground LLM generation in externally retrieved documents, improving factual accuracy compared to parametric-only generation [1]. Since then, RAG has been increasingly applied to regulated and compliance-sensitive domains.

In the clinical and pharmaceutical space, Waikar et al. [4] used RAG pipelines combined with LLMs to evaluate the regulatory compliance of drug information and clinical trial protocols against FDA guidance, reporting that RAG-based assessments aligned closely with manual expert review. Similarly, a RAG system named Raven [2] was developed to answer laboratory regulatory questions by grounding LLM responses in the relevant part of the U.S. Code of Federal Regulations, highlighting the advantages of RAG in terms of accuracy, explainability, and maintainability for regulatory question answering. In the transportation domain, Melton et al. [3] compared fine-tuned generative models, including a RAG-augmented LLaMA pipeline, for retrieving hazardous-materials transport regulations, underlining the importance of grounding for high-stakes regulatory retrieval.

Beyond document question answering, RAG-based frameworks have also been proposed for automated compliance checking rather than pure question answering: a recent framework [5] combines embedding-based and logic-based methods in a layered architecture to verify whether a business process complies with a given regulation, rather than merely answering a question about it. From a governance perspective, the privacy implications of using RAG with sensitive or regulated documents have also been examined [6], noting that RAG can, if properly designed, have a positive effect on compliance posture rather than only introducing new risks.

Across this body of work, two recurring themes are directly relevant to TextilBot: (i) RAG is consistently reported as a way to reduce hallucination and increase explainability in regulatory question answering compared to LLMs used without retrieval, and (ii) most reported systems target a single language and a single regulatory domain, with comparatively little reported on cross-lingual consistency when the retrieved source corpus and the user's query are in different languages, a gap directly addressed by the present work in the context of EU textile labelling regulation.

3. Contribution

TextilBot contributes a practical, end-to-end RAG pipeline applied to a previously uncovered regulatory domain (EU textile fibre labelling), together with a specific engineering solution to a cross-lingual consistency problem that, to the best of our knowledge, is not explicitly addressed in the regulatory-RAG literature reviewed above.

3.1 System Architecture

The system is organized into three modules: an ingestion module, an online retrieval-generation module, and a user-facing interface.

Ingestion: The official PDF of Regulation (EU) No 1007/2011 is parsed using pypdf and split into 68 overlapping chunks of 500 words with a 50-word overlap, chosen to preserve context across chunk boundaries. Each chunk is embedded using the multilingual BAAI/bge-m3 sentence embedding model and stored in Qdrant Cloud together with its source metadata.

Retrieval and generation: At query time, the user's question is embedded with the same model, and the top-5 most similar chunks are retrieved by cosine similarity. These chunks are inserted as context into a prompt sent to the LLM, which generates a grounded, citation-backed answer.

Interface: A Streamlit application exposes a chat-style interface with clickable example questions, conversation export, thumbs-up/down feedback, and a clear-conversation control. The system was first deployed with Llama 3.3 70B served via the Groq API, and later migrated to Gemini 2.5 Flash for improved quota availability and context handling; this migration only required changes in the LLM-calling layer of the retrieval-generation module, confirming the benefit of decoupling the RAG logic from the specific LLM provider.

3.2 Cross-Lingual Consistency Problem and Solution

During evaluation, we observed that a simple natural-language instruction in the system prompt (“detect the language of the question and answer in the same language”) was not reliably followed by the LLM when the retrieved context chunks were in French but the question was in English: the model would occasionally answer in French, apparently influenced by the dominant language of the injected context rather than the language of the question itself.

To address this, we adopted a three-stage detection-injection-verification strategy:

1. **Detection.** The language of the incoming question is detected programmatically (using the `langdetect` library) rather than left to the LLM to infer.
2. **Injection.** An explicit, repeated instruction tag (e.g. [MANDATORY RESPONSE LANGUAGE: ENGLISH]) is inserted directly into the user prompt, separated from the general rule set so that it is not diluted among other, lower-priority instructions.
3. **Verification.** After generation, the language of the produced answer is re-detected. If it does not match the expected language, the system retries generation once with a reinforced instruction, and as a final fallback, performs an explicit LLM-based translation of the generated answer while preserving citations and formatting.

This approach moves part of the responsibility for language consistency from the LLM's instruction-following ability to deterministic, code-level control, which proved more robust in practice than relying solely on prompt phrasing.

3.3 LLM Selection and Migration

The next fatal problem that did arise after fixing the language detection and responding and it took a long time of searching and fixing bugs was the limitation of tokens.

Initially, the system was initially deployed using Llama 3.3 70B served via the Groq api? After finishing, during development and testing, the daily token quota of the Groq free tier with 100000 tokens per day was exhausted, causing rate limit errors that interrupted the service. To resolve this, the LLM was migrated to Gemini 2.5 Flash (Google AI), which offers a significantly higher free quota, native multilingual support, and strong instruction-following capabilities.

4. Results and Discussion

Qualitative testing was conducted using a set of representative questions, including direct factual questions (example: the list of products exempt from labelling under Annex V), numerical/regulatory threshold questions (e.g., tolerance percentages for foreign fibers under Article 20), out-of-scope questions, simple greetings, and matched French/English question pairs used to assess language consistency.

Before the detection-injection-verification fix, identical questions asked in English sometimes returned partially or fully French answers, and greetings sometimes triggered a full document-referencing response instead of a short, document-free reply. After fixing those problems, both were resolved: English questions consistently produced English answers with correctly cited sources and greetings were handled deterministically via a code-level shortcut that bypasses retrieval and generation entirely.

Limitations: The evaluation presented here is qualitative rather than based on standardized RAG evaluation metrics (e.g., context relevance, faithfulness, answer correctness scores). The first limitation is the application is limited to a single regulation, and the retrieval module does not currently re-rank or deduplicate overlapping chunks beyond the top-5 cosine similarity selection. The greeting list is a static, language-specific set of strings, which may not generalize to all possible greeting phrasings.

5. Conclusion

TextilBot demonstrates that a RAG-based architecture can provide accurate, source-grounded, bilingual question answering over a dense European regulatory text in the textile domain, an application area not previously covered by the regulatory-RAG literature reviewed. The main technical contribution beyond standard RAG practice is a deterministic detection-injection-verification mechanism for cross-lingual answer consistency, addressing a failure mode, language drift toward the dominant language of the retrieved context, that simple prompt instructions failed to prevent reliably. Future work includes extending the corpus to additional EU textile regulations, introducing quantitative RAG evaluation metrics, and adding usage logging to support continuous improvement of retrieval quality.

6. References

- [1] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.
- [2] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *arXiv preprint arXiv:2005.11401*.
- [3] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.
- [4] Xiong, L., Yu, C., Tang, W., et al. (2024). BGE-M3: Multi-Functionality, Multi-Lingual, Multi-Granularity Text Embeddings. *arXiv preprint arXiv:2402.03216*.
- [5] European Parliament and the Council. (2011). Regulation (EU) No 1007/2011 on Textile Fibre Names and Related Labelling and Marking of the Fibre Composition of Textile Products. *Official Journal of the European Union*.
- [6] Google. (2025). Gemini 2.5 Flash Documentation. Google AI.
- [7] Qdrant Team. (2024). Qdrant Vector Database Documentation. <https://qdrant.tech/documentation/>
- [8] Streamlit Team. (2025). Streamlit Documentation. <https://docs.streamlit.io/>
- [9] LangChain. (2025). Retrieval-Augmented Generation (RAG). <https://python.langchain.com/>
- [10] Groq. (2025). Groq API Documentation. <https://console.groq.com/docs>